1. Your Vision of AI in Software Development

How do you see AI transforming the software development industry over the next 3-5 years?

AI is already saving a great deal of time, especially on repetitive work like boilerplate, CRUD scaffolding, and basic debugging. Over the next 3-5 years I expect MVPs and simple, common solutions to become much cheaper and faster to build. The user experience will change too. Instead of following a full booking flow, a visitor will tell an AI search box what they want. It will find the product, confirm the details, and place the order, and the user will only need to complete the payment. That will make software easier, faster, and more enjoyable to use.

But speed is not the same as reliability. The stability and quality of a system still depend on the developer behind it. AI can produce working code that is built on weak foundations, and I have seen this myself:

- **Poor data design:** When I let AI code freely as an experiment, it made first_name the primary key. Names repeat, so this breaks as soon as two users share one. A similar mistake would be a banking system that stores amounts as strings instead of numeric or decimal types, which makes calculations unreliable.
- **Weak backend validation:** Many systems validate files only on the frontend. The backend will still accept any file type of any size. On a shared hosting server I worked on, users uploading very heavy files degraded performance, and the site could go down entirely once storage and resource limits were exceeded.
- **Missing relationships and constraints:** Without proper foreign keys, indexes, and validation, data becomes inconsistent no matter how fast the code was written.

AI does not know these problems exist unless the developer does. The better the prompt, the better the output, and a better prompt comes from solid fundamentals.

Will AI replace traditional software development practices, or change the role of developers?

AI will change the role of developers, not replace them. It will replace the most basic, repetitive work, so developers must move up a level. What AI cannot replace is business logic, common sense, and real-world context. Requirements documents never capture everything: edge cases, user behavior, legal constraints, and how a business actually runs. Humans are needed to understand and decide these things. Whether or not the AI hype bubble bursts, AI will remain a powerful tool, but it will stay a tool.

AI also lets developers write less boilerplate and spend more time thinking about architecture and design. Still, developers need to build the core structure themselves. When a solid foundation is in place, AI can follow its patterns and extend it reliably. Developers with strong fundamentals also debug simple issues quickly on their own, instead of wasting time and AI tokens on problems they could solve in minutes.

What skills should developers focus on to remain competitive?

1. **Engineering fundamentals:** database design and normalization, proper data types and relationships, design patterns, and clean architecture that is not over-engineered.

2. **Scalability and performance:** caching (e.g., Redis), load balancing, queues, rate limiting, and sensible file-size and resource limits.
3. **Security:** server-side validation (never trust the frontend alone), file verification, authentication and authorization, and protection against the larger number of attacks we should expect as AI makes attacking easier too.
4. **AI integration:** connecting LLMs to real software to build smarter, more productive systems, such as conversational search, automated workflows, and AI-assisted ordering.
5. **Critical review of AI output:** treating AI-generated code like a junior developer's work, reviewing it carefully and never shipping it blindly.

In short: AI will make development faster and cheaper, but it will not replace developers who understand how systems really work. Developers should use AI to move faster while getting their own hands dirty with code and concepts, because the stronger the developer's knowledge, the better AI performs for them. FANG like systems or to build such systems it's always needed for quality developers not quality robots only where you can't accuse for any severe issue!

2. Limitations of AI in Software Development

What are the major limitations of current AI tools when developing a complete enterprise-level software system?

For example, where do you think AI still struggles with system architecture, database design, security, business logic, and long-term maintenance?

AI is excellent at generating code for a well-defined task, but an enterprise system is more than a collection of tasks. It needs consistent decisions across hundreds of files, deep knowledge of the business, and years of reliable operation. AI's main limitation is that it optimizes for code that works right now, while enterprise software must keep working under load, under attack, and under change.

AI gonna do the MVP and blind for the all business logics that human thinks. AI basically a brain build copying the human thinking patterns, so human must involve for the best output. It does not take responsibility for the consequences, so the developer has to.

Where AI still struggles:

- **System architecture:** AI tends to solve the problem in front of it without seeing the whole system. It can't reliably decide when to use caching (e.g., Redis), queues, load balancing, or service boundaries, because those choices depend on expected traffic, budget, and team size. It also drifts toward two extremes: a quick, tightly coupled solution that won't scale, or an over-engineered one with patterns and layers the project doesn't need. Without a clear structure set by the developer, each AI-generated feature can follow a slightly different style, and the codebase slowly becomes inconsistent.
- **Database design:** This is where I have seen AI fail most clearly. In one experiment where I let it code freely, it made first_name the primary key, which breaks as soon as two users share a name. Other typical problems are wrong data types (for example, storing money as strings instead of decimals in a banking system), missing foreign keys and relationships, absent indexes, and no thought for data integrity, transactions, or future growth. These flaws are invisible at first and become very expensive once real data is in the system.

- **Security:** AI often builds protection only at the surface, such as frontend validation, and forgets that the backend must enforce everything independently. For example, a server may accept any file type with no size limit. I saw this on a shared hosting server I worked on, where heavy uploads from many users degraded performance and could take the site offline. AI also commonly misses rate limiting, authorization checks, proper file verification, and safe handling of secrets, unless the developer asks for them explicitly.
- **Business logic:** Real business rules are rarely written down completely. Edge cases, exceptions, legal requirements, approval flows, and unwritten "how we actually do things" knowledge live in people's heads, not in the requirements document. AI can only work with what it is told, so it fills the gaps with assumptions that look reasonable but are wrong for that business. Common sense and domain understanding remain human strengths.
- **Long-term maintenance:** AI does not remember a project the way a team does. It has limited context, so it can miss how a change in one module affects another, duplicate existing logic, or ignore earlier decisions. Over time this produces technical debt, inconsistent patterns, and code that nobody fully understands. Debugging production issues, handling migrations on live data, managing dependencies, and keeping the system stable over the years still require engineers who understand the whole picture.
- **Accountability and verification:** AI can sound very confident even when it is wrong, and it will not warn you about a flaw it does not know exists. It also cannot be held responsible if a system fails or data is breached. Everything it produces must be reviewed, tested, and understood by a human.

How to work around these limitations:

The best approach is to treat AI as a fast but inexperienced junior developer. The developer defines the architecture, database schema, and security rules first, then lets AI follow that structure and handle repetitive work. Every output is reviewed and tested before it ships. The more solid the developer's engineering knowledge is, the better the prompts, and the better the results.

Basically, AI is a powerful accelerator, but it still lacks big-picture architectural judgment, deep understanding of data and security, real-world business knowledge, and long-term ownership. Those gaps are exactly why skilled developers remain essential for enterprise-level systems. So the developer must onboard the AI with the project properly, have to huge works by himself and then it’s AI which gonna make it faster. You can’t ask AI or just say: ‘Hey build be a health insurance website where I will have lots of users, doctors and medical reports will be generated by doctors’; this gonna make much hallucinations, So you need to code lot by own, fix the database, load, Redis, Rate limiting, Design pattern, security concern and let not AI handle them, they just do the coding part, all other stuffs the programmer must handle.

3. Handling Legacy Systems and Historical Data

Suppose we receive a project from a company that has been operating for the last 10 years using the same business processes and software patterns.

The company has a large amount of historical data, and maintaining data accuracy and business continuity is essential.

As a developer, how would you approach this project?

- How would you understand and document their existing business processes?
- How would you handle existing databases and historical data?
- How would you plan data migration without losing important information?
- How would you ensure the new system works without disrupting existing operations?

➔ A company that has run on the same processes for 10 years has two things worth protecting above everything else: its data and its business continuity. So my approach is to be careful first and fast later. I will not touch anything until I have a safe copy and a clear understanding of how the system actually works.

Step 0: Protect what exists

The very first thing I do is take a complete backup of everything: the source code, files, uploads, configuration, and a full database dump. The code goes into a GitHub repository as a backup, and the database dump is stored separately in a secure location. I protect the main branch so it cannot be changed directly. It always remains the exact copy of what the company handed over, so there is always something to fall back to.

All new work happens on separate branches: a development branch for building and testing changes, and a staging branch for final verification before anything goes to the live server. Changes are committed in small, clear steps, and I keep the README updated at all times with setup steps, environment details, and what has changed. That way nothing depends on my memory.

How would you understand and document their existing business processes?

I would not rely only on reading code. I would do three things together:

- **Run the system in Docker:** I containerize the existing project so it runs in the same environment as the live server. This lets me explore it safely, see how it behaves, and avoid the “works on my machine” problem.
- **Go through the system feature by feature:** I check the main modules one by one, see what each does, and document it: the inputs, the outputs, the user roles, and the flow from start to end.
- **Talk to the people who use it daily:** After 10 years, many rules exist only in the staff’s habits, such as special discounts, manual approvals, and exceptions. These are rarely written anywhere, and I confirm them with the business team before I assume anything.

I record all of this in clear written documentation. I might also record a screen walkthrough of the system, so when a new team member joins, they can understand the project in much less time instead of needing repeated explanations.

How would you handle existing databases and historical data?

First, I study the database without changing it: the tables, relationships, data types, indexes, and any missing constraints. Old systems often have problems such as inconsistent data, duplicate records, missing relationships, or wrong data types. I list these issues but I do not fix them blindly, because the 10 years of history has to remain accurate.

I work on a copy of the database in my development environment, not on the live one. Only test or sample data is kept there, and sensitive information is masked or removed where possible. This way I can experiment freely with no risk to real records.

How would you plan data migration without losing important information?

The key idea is that I do not replace the old database with a new one. I make incremental, controlled changes to it.

- Every change is written as a **migration** and tested first in the development environment.
- Once it is verified, I run **only the specific, tested SQL changes** on the VPS. The existing data stays where it is, and only the required structure changes. The whole database is never taken down and replaced.
- Before each production change, I take a fresh backup and prepare a **rollback plan**, so I can return to the previous state quickly if something goes wrong.
- After each change, I **verify the data**: record counts, totals, and key relationships should match before and after.

If a bigger restructuring is needed, I do it in small steps rather than one big move, and I check the results at every step.

How would you ensure the new system works without disrupting existing operations?

- **Staging first:** Everything is tested on the staging branch in an environment that mirrors production before it is released.
- **Small, gradual releases:** Instead of one big launch, I release in small parts so any issue is small and easy to find and fix.
- **Run in parallel when needed:** For critical features, the old and new behavior can run side by side for a period, and I compare the results to confirm they match.
- **Low-traffic deployment windows:** Updates are scheduled when the business is least busy, with a rollback ready.
- **Monitoring after release:** I watch logs and errors closely after each deployment, so problems are caught before users report them.
- **Keeping the team informed:** Staff are told what is changing, so nothing surprises them in their daily work.

I protect the original first, understand the system and its hidden business rules properly, document everything, and then change it in small, tested, reversible steps. The old data stays safe, only the needed changes are applied, and the business keeps running without interruption. I also will ensure there won't be any single point of failure in the system for the updates.

4. Your AI-Powered Development Workflow

You mentioned that you use AI coding tools in your development workflow.

We want to understand how you actually use AI in your daily development activities.

- Which AI tools do you currently use?
- At which stages of development do you rely on AI the most?
- How do you verify AI-generated code?
- How do you ensure code quality, security, performance, and maintainability?
- Can you share an example where you rejected or significantly modified AI-generated code because it was technically incorrect?

➔ Which AI tools do you currently use?

I use **Claude, DeepSeek, and Gemini** for reasoning, code review, and problem-solving, and **Antigravity IDE** most heavily, since it works directly inside my project and codebase. I don't treat them as interchangeable. If one tool gives a weak or doubtful answer, I cross-check it with another, and the final decision always stays with me.

At which stages of development do you rely on AI the most?

- **Boilerplate and repetitive work:** CRUD, forms, validation rules, Blade/React components, and API resources.
- **Understanding code:** Explaining unfamiliar code, and generating an ERD to see a complex database quickly.
- **Debugging:** Reading error logs and stack traces and suggesting where the logic might be wrong.
- **Tests, documentation, and test data:** First drafts of test cases, README sections, and seeders.

I rely on it **least** for database design, architecture, and security-critical logic. I plan and build those parts myself first, and then AI follows my structure.

How do you verify AI-generated code?

I never merge code I haven't understood. My process:

1. **Read it piece by piece** and check that it matches the design pattern and structure I set.
2. **Run it in my Docker dev environment** with test data, never directly on live.
3. **Test the edge cases**, not only the happy path: different roles, empty data, repeated actions, and "what changes if I edit one record?"

4. **Check the database effects:** the queries it runs, the relations it uses, and whether it touches data it shouldn't.
5. **Commit in small steps** on a development branch, so a bad AI change is easy to review and revert.

How do you ensure code quality, security, performance, and maintainability?

- **Quality and maintainability:** I set the structure first, so AI-generated code follows the same style. I keep a clear README and make small, meaningful commits.
- **Security:** I check that the backend validates everything on its own: server-side validation, file type and size limits, authorization checks, and rate limiting. AI often skips these unless asked. I also never paste secrets or real user data into AI tools.
- **Performance:** I check for N+1 queries (using eager loading), missing indexes, and heavy work that should go to a queue. I use Redis for frequently read data.
- **Review before release:** Code goes through development, then staging, and only then to live.

Can you share an example where you rejected or significantly modified AI-generated code?

In a **school management SaaS** project, I used AI to build the **class routine system**. The requirement was to generate routines for **all classes, all sections, and all groups**. The AI only built it for classes and ignored sections and groups. The code looked complete and ran without errors, but it did not match the real structure of the school, so it was wrong at the business-logic level. I rejected that approach and redid a large part of it myself, starting from the correct data relationships between class, section, and group.

Later, a second problem appeared in the **teacher (tutor) assignment**. When I changed one specific class in one specific shift, the same teacher's routines for **other classes on that day also changed**. The AI had updated records by teacher and day only, without scoping the update to that class, section, and shift. That would have silently corrupted the timetable of the school. I fixed it by scoping every update to the exact routine record and adding a **conflict check**, so the same teacher can't be double-booked in the same period.

What I learned from it: AI follows the words of the prompt, not the unwritten business rules. I now describe the full hierarchy and constraints in the prompt first (for example, "class → section → group → shift"), and I test with the "change one thing and check what else changed" approach before accepting any such feature.

5. AI vs. Developer Collaboration

Suppose you and another experienced developer are assigned to build the same complete software system. Both developers have access to the same AI tools and are instructed to use AI as much as possible.

How would you differentiate your development approach from the other developer?

What strategy would you follow to build a better system in terms of:

- System architecture
- Code quality and maintainability
- Database optimization
- Security
- Scalability
- Development speed
- Long-term product maintenance

➔ If both of us have the same AI tools and the same instruction to use AI as much as possible, the difference is not the tool but how it is used. A developer who prompts straight away gets code fast, but it follows whatever assumptions the AI makes. I don't rush with AI. I first understand the project and what it asks for, study what it needs, and organise everything. Then I build the main structure myself: the folder structure, the design pattern, the database relations, and the authentication approach. After that I prompt the AI, and because the prompt is precise, the output is stable and reliable. AI follows my structure instead of inventing one. I also review the generated code piece by piece before I confirm and proceed, and I never merge code I haven't understood.

What strategy would you follow to build a better system?

System architecture

I study the requirements and follow the design pattern the project already uses, or choose the most suitable one for the features I need to build (for example Repository or Service layer in Laravel). I choose it deliberately, not by whatever the AI suggests, and I keep it simple and not over-engineered. For APIs, I first decide which authentication pattern fits: **Sanctum** for SPAs and mobile apps with simple token auth, **Passport** when full OAuth2 is needed (third-party access), or **JWT** for stateless, cross-service setups. I then apply that one consistently.

Code quality and maintainability

Because the structure is already set, every AI-generated feature follows the same style and the codebase stays consistent. I keep a proper README (setup, environment, and decisions) and make small, meaningful Git commits so history is easy to follow. Any new developer can pick up the project without depending on me.

Database optimization

This is the part where I use AI the least. I design the tables, relationships, data types, foreign keys, and indexes myself, because mistakes here are the most expensive to fix later. To understand a complex database quickly, I ask AI to generate an **ERD** (Entity Relationship Diagram) so I can see the

relations at a glance. Then I write and review the **DDL**, **DML**, and **DQL** (the structure, the data changes, and the queries) carefully, checking query performance and indexes before the final go.

Security

I make sure the backend enforces everything independently and never trusts the frontend alone: server-side validation, file type and file size limits, proper authentication and authorization, and rate limiting on sensitive endpoints. I check each of these in AI-generated code, because AI often skips them unless asked.

Scalability

- **Redis caching** for frequently read and expensive data, so the database isn't hit repeatedly.
- **Queue jobs** for heavy work like emails, notifications, and SMS, so the user gets a fast response and the system stays consistent under load.
- **Docker** so the whole team works in the same environment as production, and the system can be scaled and deployed the same way everywhere.

Development speed

I use AI where it saves the most time: boilerplate, CRUD, repetitive code, test data, and first drafts. The upfront planning costs a little time at the start, but I save much more later, because I don't waste time and AI tokens on rework and debugging problems caused by a weak foundation. Overall, I am faster where it counts.

Long-term product maintenance

A clean structure, consistent patterns, documentation, clear commits, and a reliable Docker environment make the product easy to maintain, fix, and extend for years. Because I understand every important part of the code, I can find and solve production problems quickly instead of depending on AI to explain my own system to me.

The other developer may get a working system faster in the first days, but I will deliver a system that is stable, secure, scalable, and easy to maintain, because I make the key engineering decisions myself and use AI to execute them quickly.

6. Real-World Problem-Solving and Ownership

You mentioned SugarPros as one of your most challenging projects.

Suppose the platform suddenly starts experiencing inconsistent booking records, duplicate payment transactions, and failed pharmacy API synchronization in production.

How would you investigate and resolve these issues?

Please explain your debugging strategy, database consistency approach, error-handling mechanism, and how you would prevent similar problems in the future.

➔ My first rule is to protect users and data before I start fixing. Inconsistent bookings, duplicate payments, and failed pharmacy sync all touch money and patient care, so I act in this order: contain, reproduce, find the root cause, fix, repair the data, and then release carefully.

Step 1: Reproduce the problem and contain it

- I use my test account on the live system and run a test booking, following the same steps the customers followed, to see the problem myself. I use the payment gateway's test/sandbox mode for this, so no real money moves.
- I then disable the affected feature (booking, or the payment/pharmacy-sync part) immediately, using a feature flag or a "temporarily unavailable" message. Only that feature is paused and the rest of the platform keeps running, such as messaging, login, and the dashboard. If duplicate charges are still happening, I also pause the related queue workers.
- Before I touch any data, I take a fresh database backup.

Step 2: Debugging strategy

- **Read the evidence first:** I check the Laravel logs, the failed_jobs table, and the payment and pharmacy API logs, and I trace one booking across them using its booking ID or transaction reference.
- **Check one micro-feature at a time in the dev environment:** slot selection, booking creation, payment, status update, and pharmacy sync. This shows exactly where the flow breaks, and I fix that bug.
- **Use AI as a second pair of eyes:** I paste the relevant controller or service code together with the error log and ask where the logical error might be. If AI doesn't solve it, I go deeper and debug it myself. I review every AI suggestion before applying it.

Step 3: Duplicate payments and database consistency

I check first whether the payment function is being called twice: a double-click, a retried queue job, a payment webhook delivered twice, or two requests arriving at the same time. Then I make the flow safe:

- **DB::transaction()**, so the payment record, booking status, and related updates all succeed or none do.
- **A unique index on the transaction reference**, so the database itself rejects a duplicate even if the code has a bug.

- **A lock on the booking/payment row** (lockForUpdate()), so two simultaneous requests can't both pass the "not paid yet" check.
- **An idempotency check** in the payment and webhook handlers: if this transaction was already processed, ignore it.

I also repair the existing data: compare bookings against payments and the gateway's records, list the mismatches, fix them after review, and refund real duplicate charges through the gateway.

Step 4: Pharmacy API and error handling

- I re-read the pharmacy API documentation and compare it with my request: the endpoint, the required fields, the data format, and the authentication method. If required data is missing from the POST request, I add it.
- I check the response status codes and error messages to see whether the cause is a validation error, an expired token, a timeout, or a changed response format.
- I test with test data first, then do a live check with my test account.
- The sync runs in a **queue job** with limited retries, backoff, and a timeout. If it still fails, it lands in failed_jobs with an alert, and the prescription is marked "pending sync" so nothing is lost silently.

Step 5: Release and prevent it from returning

- I push the fixes one by one, verifying each on staging and then live, so the system stays stable. I re-enable the features for customers only after everything is confirmed working.
- I add tests for double-submit payments, duplicate webhooks, concurrent bookings, and API failures.
- I add monitoring and alerts for failed jobs and payment errors, plus a daily reconciliation check between bookings, payments, and pharmacy sync.
- I write down the root cause and the fix in the README, so the knowledge stays in the project.